

IFQ557: Assignment 2 Submission Report

FuturePath - Nonprofit Education Event Management System

Section 1: Team declaration

Option A - Functional Team

Functional Team		
Group/team number	Group 07	
GitHub or Gitee name and link	https://github.com/7000frank/futurepath-event-management	
Full name	Student #	Email
Frank	2422441010	2268109713@qq.com
Chen	2422441008	lv13913119139@outlook.com
George	2422441020	yonghaopu67@gmail.com

Application overview

FuturePath upgrades the selected Assignment 1 prototype into a working Flask and SQLite application for free Gaokao guidance events. It provides dynamic event discovery, score-based recommendations, event details, comments, authentication, role-based event management, one-place bookings, booking history and automatic event states.

Technology: HTML, CSS, Bootstrap, Python, Flask, Flask-WTF, WTForms, Flask-SQLAlchemy, SQLAlchemy, Flask-Login, Bootstrap-Flask, Werkzeug and SQLite.

Section 2: Implementation summary

Assessment area	Implemented evidence	Verification
Landing page and search	Database-backed event cards; keyword, category and status filters; score finder; live state and remaining-place displays.	Browser QA
Event details and comments	Public event detail pages show image, description, date, time, venue and presenter. Logged-in users can post dated comments.	Automated tests
Event management	Teacher accounts create events. Only the event owner can update or cancel. Open, Sold Out, Inactive and Cancelled states are calculated automatically.	Permission tests
Bookings	Authenticated users reserve one free place, receive a unique reference, view history, cancel and rebook. Capacity and Sold Out state update from active bookings.	Booking tests
User management	Registration captures first name, surname, email, password, contact number, street address and Student/Teacher role. Password hashing, login, logout and route-level access control are implemented.	Auth tests and browser QA
Errors and validation	Custom 404/403/500 pages, CSRF-protected forms, WTForms validation, capacity checks, ownership checks and friendly flash messages.	Error tests

Database design and supplied data

The SQLite database contains User, Event, Booking, Comment and Category tables with foreign-key relationships. User records include first name, surname, email, password hash, contact number, street address and role. The submitted database includes synthetic teacher and student accounts, events, bookings and comments for marker testing.

Testing status

Thirteen automated unittest cases pass. They cover registration including the required street address, login/logout, role permissions, search, comments, creation and ownership, booking uniqueness, cancellation and rebooking, capacity/status behaviour, protected deletion, and custom 404/500 responses. Desktop and 390-pixel mobile browser checks found no horizontal overflow or console errors.

Section 3: Task contribution table

Task/workstream	Contributor(s)	Completed work
Prototype and visual direction	Frank	Selected and adapted the FuturePath Assignment 1 prototype; maintained the nonprofit theme and content direction.
Flask application structure	Frank	Application factory, blueprints, routes, templates, forms, static assets and package configuration.
Database and seed data	Frank / Chen	Frank designed the SQLAlchemy schema, relationships, calculated capacity/status properties and synthetic data. Chen implemented the required street-address field on a feature branch; Frank reviewed and integrated the approved parts without importing the database-conflict commit.
Authentication and roles	Frank / Chen	Frank implemented registration, password hashing, login/logout, Student/Teacher roles and access-control decorators. Chen contributed the street-address registration field and related test coverage.
Discovery and event pages	Frank	Search, category/status filters, score finder, event details, comments and responsive presentation.
Event management	Frank	Teacher creation workflow, owner-only update/cancellation, automatic event status, validation, management dashboard and protected permanent deletion.
Booking workflow	Frank	One-place booking, unique references, duplicate prevention, history, cancellation, rebooking and Sold Out calculation.
Testing and error handling	Frank / George	Automated tests, custom errors, browser QA, mobile checks and edge-case verification.
Report and submission package	Frank; Chen / George reviewing	Declaration, contribution evidence, AI statement, final review and ZIP assembly.

Section 4: Individual contribution statements

The following statements summarise each team member's contribution to the completed application.

Frank	Frank led FuturePath from prototype selection through final delivery. He designed the Flask package and SQLAlchemy schema, implemented event discovery, score matching, details, comments, authentication, roles, booking and rebooking rules, event ownership, the teacher management dashboard, safe deletion and automatic statuses. He also developed the responsive interface, supplied the database and media, wrote most automated tests, resolved integration issues, reviewed pull requests, configured main-branch protection, prepared the report and assembled the final submission package.
Chen	Chen contributed a focused registration and persistence improvement on the chen-street-address feature branch. He added the task-sheet-required street-address field across the WTForms registration form, User model, registration template, seed accounts and automated registration tests. He also investigated a SQLite merge conflict and retained the appropriate local database version on his branch. Frank reviewed the feature before integrating the approved source changes into the protected main branch.
George	George supported requirements review, quality assurance and submission preparation. He checked the implemented workflows against the brief and rubric, reviewed registration and role behaviour, exercised representative event, booking and error-handling scenarios, and reported usability observations for the team to resolve. He also reviewed the demonstration sequence and final documentation for clarity and consistency. His contribution focused on verification and coordination rather than core application code.

Section 5: git shortlog output

```
git shortlog -sne --all
2  Chen <lv13913119139@outlook.com>
2  Frank <2268109713@qq.com>
1  7000frank <2268109713@qq.com>
Frank's two Git identity labels represent the same contributor (3 commits in total).
```

This evidence was captured on 4 July 2026 before the final packaging commit. The feature-branch and main-branch histories below provide additional contribution evidence, and the submitted code was reviewed against the assignment requirements before packaging.

Figure 1. Chen feature-branch commit history.

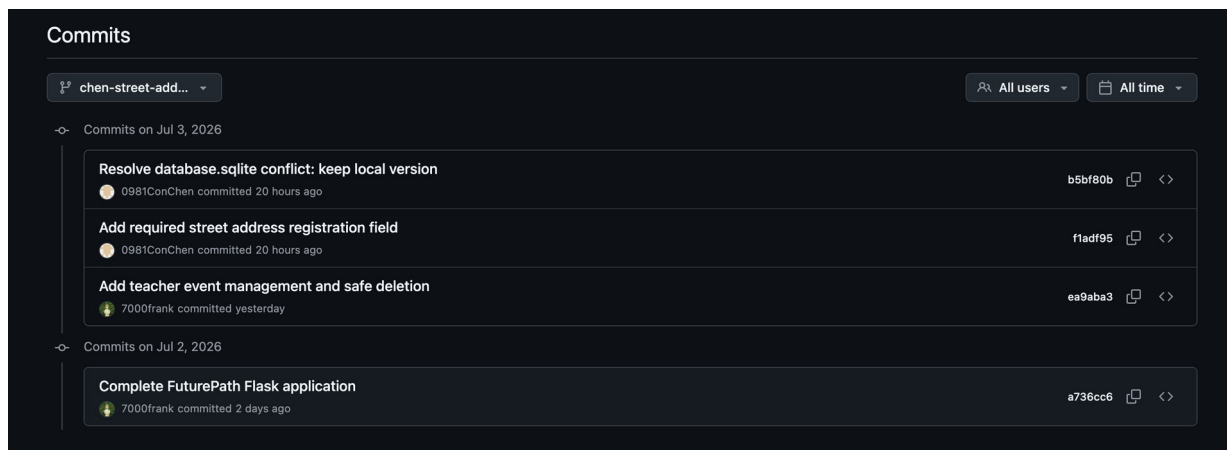
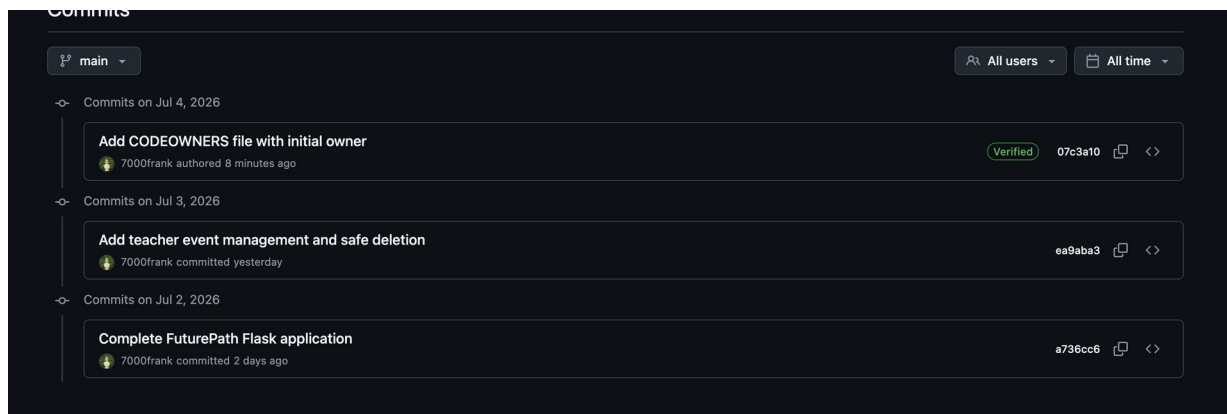


Figure 2. Protected main-branch commit history.



Section 6: Team use of AI statement

Part B - AI was used in an authorised way

As a team, we report that AI tools were used in an authorised way. AI supported interpretation of the brief and rubric, drafting and refinement of Flask code, debugging of booking and permission workflows, interface and responsive-design review, test creation, and preparation of documentation. Team members remained responsible for reviewing the generated output, checking it against the supplied requirements, running the application and tests, and deciding which changes to retain.

Representative AI use example 1

Prompt	Review the user-facing messages in FuturePath and suggest clearer wording for registration, login, booking, event management and error pages. Do not change the underlying application logic.
Input/context	Existing Flask templates, WTForms validation messages, flash messages and the nonprofit education-event context.
AI output	Suggested wording for form validation errors, booking confirmations, permission warnings, event status messages and custom 404/500 pages.
Team review	The team checked every message against the actual application behaviour, revised the wording where necessary and retained only messages that were accurate and consistent with the FuturePath tone.

Representative AI use example 2

Prompt	Suggest a Python unittest structure, synthetic test data and an edge-case checklist for registration, login, permissions, bookings, comments and event management.
Input/context	The existing Flask routes, SQLAlchemy models, WTForms forms and the functional requirements from the Assignment 2 brief.
AI output	A draft test structure and test-data examples covering duplicate bookings, Sold Out events, cancellation and rebooking, unauthorised event updates, comments, form validation and custom error pages.
Team review	The team reviewed and adjusted the assertions, used a temporary SQLite database, ran all tests locally and investigated failures. Thirteen final automated tests passed without modifying the submitted demonstration database.

Representative AI use example 3

Prompt	Review the existing FuturePath interface and suggest minor responsive CSS improvements for desktop and mobile screens without changing the established visual design.
Input/context	The existing Bootstrap templates, custom stylesheet, interface screenshots, brand colours and desktop/mobile browser results.
AI output	Suggested adjustments to spacing, form stacking, navigation behaviour, image sizing, button alignment, focus states and horizontal-overflow prevention.
Team review	The team inspected the website at desktop and 390-pixel mobile widths, checked the registration and event pages, and retained only adjustments that matched the existing design. No horizontal overflow or browser console errors remained.